

# FROZEN INTERFACE CONTRACT

Frozen interface contract standard

| Document | Version | Status | Language |
|----------|---------|--------|----------|
| ES-06    | 1.0     | FROZEN | English  |

**SSIS FRC Team 10951**

Author: SSIS | Mentor: SSIS

## 1. Purpose and Scope

The Frozen Interface Contract defines how interfaces are designed, reviewed, approved, and frozen throughout FRC Java Coding Lab 7.0. It applies to DriveIO, GyroIO, VisionIO, IntakeIO, ShooterIO, ElevatorIO, and every future hardware interface.

## 2. Core Principles

- An interface defines capability, not implementation.
- An interface remains vendor-independent and contains no REV, CTRE, or other vendor-specific types.
- An interface contains no business logic, telemetry, NetworkTables, or command coordination.
- Hardware observations flow through an Inputs Snapshot.
- The subsystem consumes the interface and owns mechanism state.
- Every breaking change requires a new architecture review.

## 3. Standard Contract Structure

```
MechanismIO
+ MechanismIOInputs
+ updateInputs(inputs)
+ output methods
+ stop()

Implementations:
MechanismIOReal
MechanismIOSim
MechanismIOAlternativeVendor
```

## 4. Mandatory Data Flow

```
Hardware
↓
IO Implementation
↓ updateInputs(...)
Inputs Snapshot
↓
Subsystem
↓
Telemetry / Commands
```

## 5. Interface Rules

- Declare only operations the subsystem truly requires.
- Method names clearly describe an action or data request.
- Units are explicit in field names or JavaDoc.
- Never return vendor objects.
- Never read DriverStation, NetworkTables, RobotContainer, or CommandScheduler.
- Never publish telemetry.
- Never store long-lived mechanism-level state in the interface.
- stop() places outputs in a safe state.

## 6. Inputs Snapshot Rules

- Each mechanism has one dedicated Inputs class.
- Inputs represent hardware observations for one periodic cycle.
- Field names are explicit, consistent, and unit-aware.
- Inputs contain no control logic.
- Inputs never call vendor APIs.
- The IO implementation fully updates Inputs.
- The subsystem does not read hardware outside Inputs.

## 7. Ownership

| Component         | Owns                              | Must Not Own                        |
|-------------------|-----------------------------------|-------------------------------------|
| IO Interface      | Hardware contract                 | Business logic, telemetry           |
| IO Implementation | Vendor API, sensor reads, outputs | Command scheduling, mechanism state |
| Inputs Snapshot   | One-cycle observations            | Control, publishing                 |
| Subsystem         | Mechanism state, safety, behavior | Vendor-specific wiring              |
| Telemetry         | Snapshot publishing               | Hardware access, control            |

## 8. DriveIO Contract Example

```
DriveIO
class DriveIOInputs
  leftPositionMeters
  rightPositionMeters
  leftVelocityMetersPerSecond
  rightVelocityMetersPerSecond
  leftAppliedOutput
  rightAppliedOutput
  leftCurrentAmps
  rightCurrentAmps
  leftConnected
  rightConnected

  updateInputs(DriveIOInputs inputs)
  setTankOutputs(double leftOutput, double rightOutput)
  stop()
```

## 9. Forbidden Anti-Patterns

- IO publishes NetworkTables or Shuffleboard.
- IO schedules or cancels commands.
- Commands call SparkMax, TalonFX, or vendor sensors directly.
- Subsystems recreate vendor hardware instead of receiving an IO dependency.
- Inputs Snapshots contain control code or API calls.
- Interfaces change casually between lessons.
- Interfaces include methods that exist only for temporary demos.

## 10. Freeze Process

❑ Backbone Check PASS.

- ☐ The interface is small and responsibility-focused.
- ☐ The Inputs Snapshot is complete.
- ☐ Method signatures and units are finalized.
- ☐ No vendor dependency exists.
- ☐ No telemetry dependency exists.
- ☐ Architecture Review PASS.
- ☐ Real and simulation implementations are feasible.
- ☐ Version is set to 1.0.
- ☐ Status is set to FROZEN.

## 11. Change Policy

After FROZEN status, new methods should remain backward-compatible whenever possible. Renaming methods, changing types, changing units, or removing fields is a breaking change and requires formal review, a version update, and a migration plan.

## 12. PASS Criteria

- The subsystem uses the interface without knowing the vendor.
- A simulation implementation can replace real hardware without subsystem changes.
- Telemetry reads only valid snapshots exposed by the subsystem or IO boundary.
- The build passes and the verification plan is confirmed.

## Revision History

| Version | Date       | Status | Notes           |
|---------|------------|--------|-----------------|
| 1.0     | 2026-07-18 | FROZEN | Initial release |